

ARCHITETTURA DEL PROGETTO: ECLETTICI.IT

Documento Tecnico di Riferimento | Corso: Cloud Native & Applied AI Engineering

Il presente documento descrive la struttura architettuale propedeutica e sequenziale per la modernizzazione ed evoluzione della piattaforma **eclettici.it** in un'infrastruttura Cloud Native integrata con logiche di Applied AI (Retrieval-Augmented Generation).

Fase 1: Inizializzazione Repository e Versioning (GitHub)

Obiettivo: Configurazione del controllo di versione centrale per l'isolamento del codice sorgente e delle configurazioni infrastrutturali.

- **Struttura:** Setup di un'architettura monorepo per la gestione coordinata di backend, frontend e file di configurazione DevOps.
- **Configurazione:** Definizione del file `.gitignore` specifico per l'ambiente Debian, IDE IntelliJ IDEA (cartella `.idea/`) e build tool Java (Maven/Gradle).
- **Dipendenze:** Nessuna. Rappresenta la base per ogni tracciamento successivo.

Fase 2: Setup dell'Infrastruttura Locale (Docker)

Obiettivo: Creazione di un ambiente di sviluppo locale standardizzato, isolato e riproducibile.

- **Servizi standard:** Configurazione del file `docker-compose.yml` per l'istanziamento di PostgreSQL come database relazionale core.
- **Estensioni:** Abilitazione dell'estensione `pgvector` all'interno del container PostgreSQL per supportare la memorizzazione e la ricerca di vettori ad alta dimensionalità.
- **Networking:** Definizione di una `docker network` isolata per la comunicazione sicura inter-container.
- **Dipendenze:** Richiede il repository inizializzato nella Fase 1 per il salvataggio dei file di configurazione IaC.

Fase 3: Sviluppo Core Backend (Java/Spring Boot)

Obiettivo: Implementazione della logica di business e delle API RESTful per la gestione dei corsi (CCNA, Python, Java).

- **Configurazione:** Scrittura del file `application.yml` per agganciare i servizi esposti dal container PostgreSQL locale.
- **Persistenza:** Implementazione dei layer di persistenza tramite Spring Data JPA (Entità, Repository).
- **Sicurezza:** Configurazione di Spring Security per l'autenticazione stateless basata sull'emissione e verifica di JSON Web Tokens (JWT).
- **Dipendenze:** Richiede l'infrastruttura di persistenza attiva e funzionante configurata nella Fase 2.

Fase 4: Integrazione Layer AI (Architettura RAG)

Obiettivo: Evoluzione del backend tramite funzionalità di intelligenza artificiale applicata basata su dati proprietari.

- **Librerie core:** Integrazione di Spring AI o LangChain4j nel file di configurazione delle dipendenze (`pom.xml` / `build.gradle`).
- **Data Ingestion Pipeline:** Sviluppo di componenti per il chunking semantico e la generazione di embedding relativi ai testi ufficiali Cisco CCNA e alle trascrizioni dei corsi Java/Python.
- **Interrogazione:** Implementazione di algoritmi di ricerca per similarità (Cosine Similarity o Euclidean Distance) su `pgvector` e orchestrazione dei prompt verso Large Language Models esterni per fornire risposte contestualizzate agli utenti.
- **Dipendenze:** Richiede il backend Spring Boot (Fase 3) e l'istanza del database vettoriale (Fase 2).

Fase 5: Automazione CI/CD e Deploy (GitHub Actions & Debian)

Obiettivo: Automazione del ciclo di rilascio ed esecuzione del deployment nell'ambiente di produzione sul server Debian.

- **Pipeline:** Scrittura di workflow GitHub Actions (`.github/workflows/`) per l'esecuzione automatica di test unitari, compilazione dell'applicazione Java e costruzione delle immagini Docker multi-stage.
- **Registry:** Push delle immagini verificate su GitHub Container Registry (GHCR).
- **Server Debian:** Configurazione di Nginx o Traefik come Reverse Proxy per la gestione dei domini `eclettici.it` e l'automazione dei certificati SSL (Let's Encrypt), accoppiati all'avvio automatico dei container aggiornati.
- **Dipendenze:** Richiede l'intero stack applicativo e i file di configurazione dockerizzazione completati e validati localmente nelle fasi precedenti.